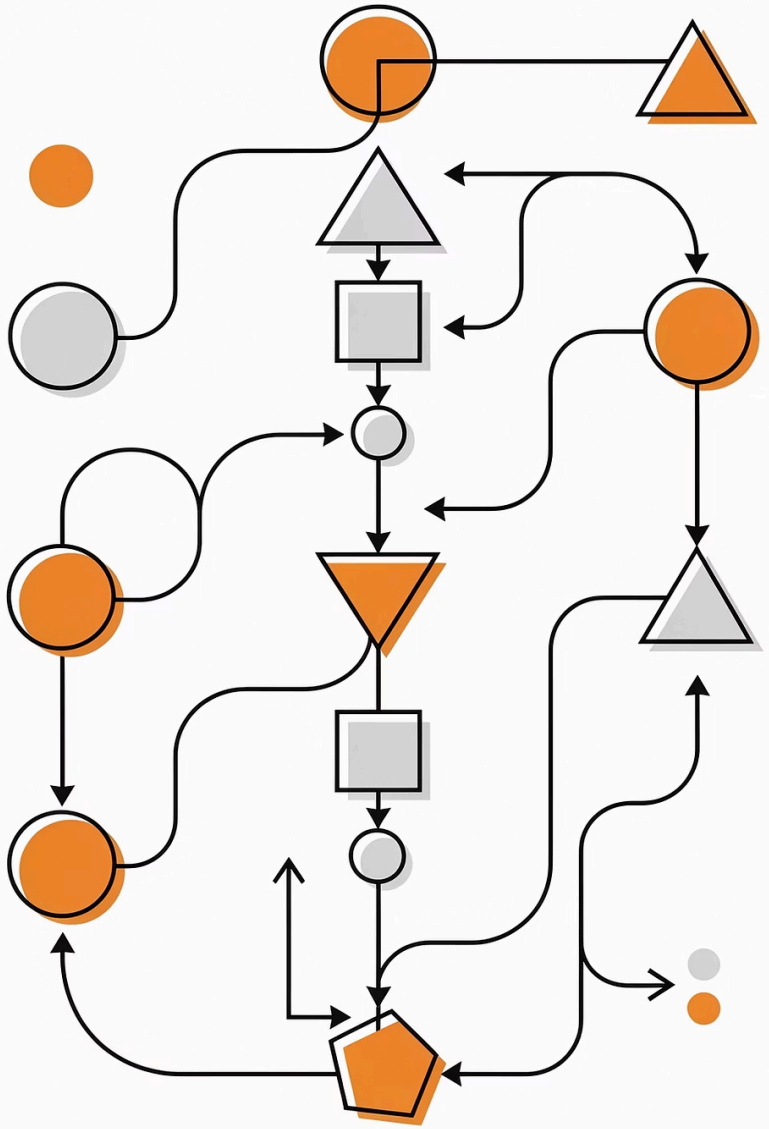
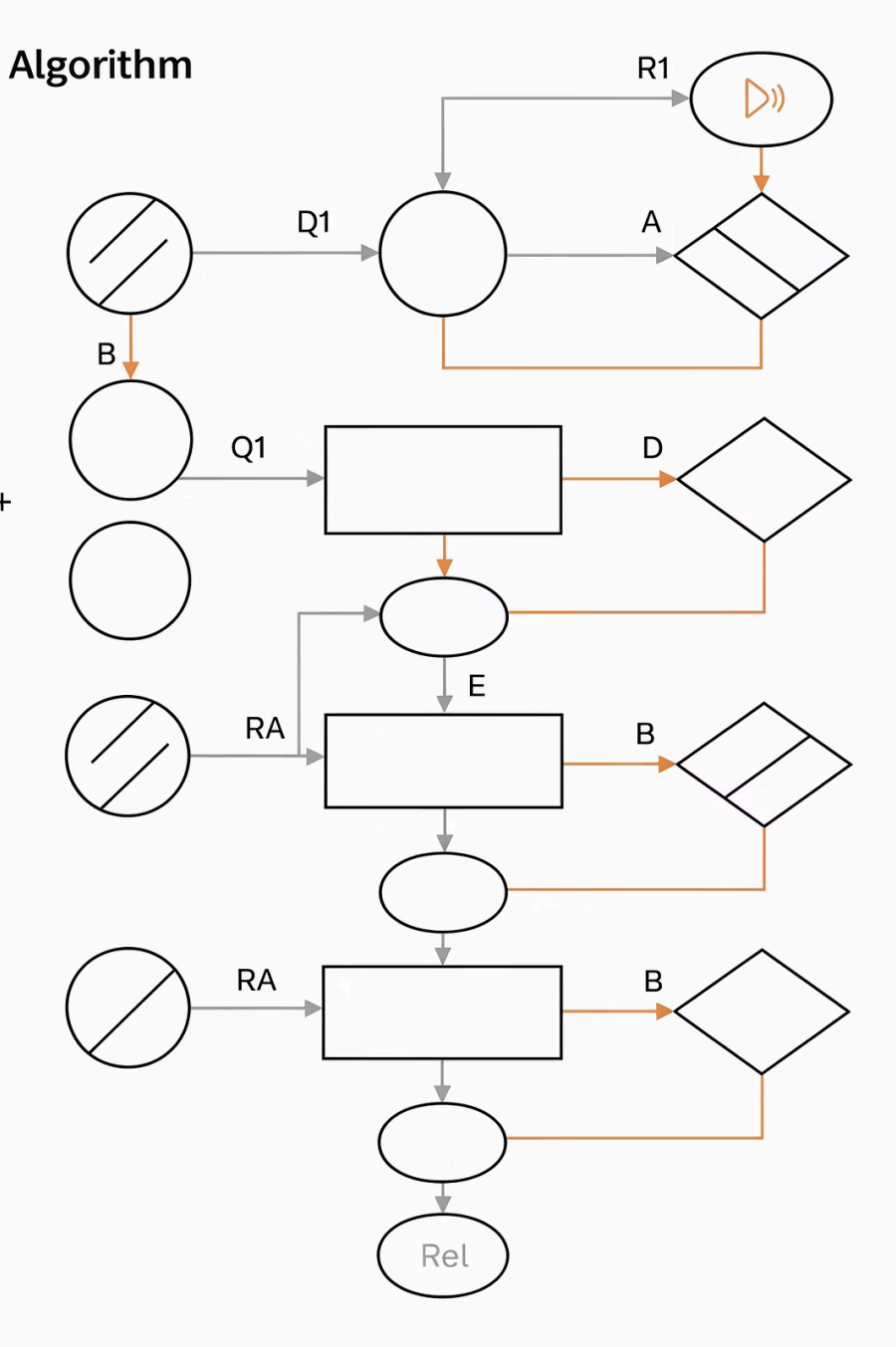


Cenni di Algoritmi e Strutture Dati

Lezione 1 – Algoritmi e BigO notation



FONDAMENTI

Cos'è un algoritmo?

Un algoritmo è una descrizione ad alto livello dei passi di una procedura per la risoluzione di un insieme di problemi. La sua implementazione può essere eseguita su un computer fisico, utilizzando **memoria** e impiegando del **tempo**.

Pensate a un algoritmo come a una ricetta: una sequenza precisa di istruzioni che, se seguita correttamente, porta a un risultato desiderato. Come una ricetta culinaria descrive come trasformare ingredienti in un piatto, un algoritmo descrive come trasformare dati di input in output desiderati.

Le competenze fondamentali

Comprensione del problema

Capire il problema che si vuole risolvere e identificare le sue caratteristiche fondamentali

Stima dell'efficienza

Imparare a stimare l'efficienza computazionale di un algoritmo in termini di tempo e di memoria

Tecniche algoritmiche

Studiare tecniche algoritmiche basiche e strutture dati differenti per risolvere problemi che condividono la stessa strategia risolutiva

Queste tre competenze costituiscono il fondamento dello studio degli algoritmi e vi accompagneranno per tutto il corso.

CASO DI STUDIO

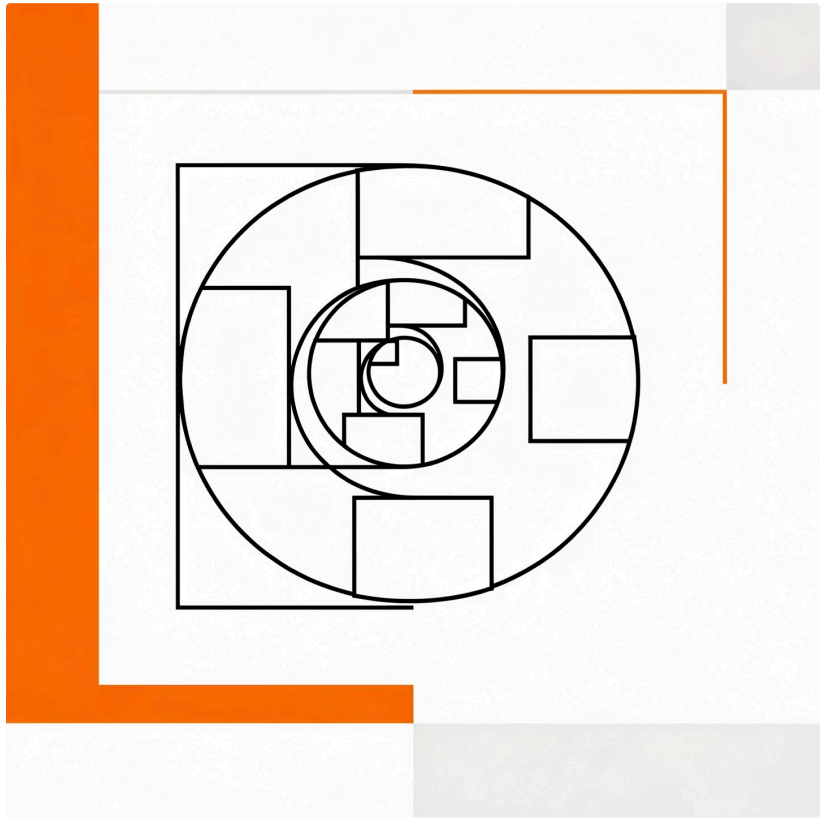
Il Problema di Fibonacci

La sequenza di Fibonacci è una sequenza numerica $F_1, F_2, \ldots, F_n, \ldots$ in cui ogni elemento è la somma dei due numeri precedenti.

Per convenzione, $F_0 = 0$ e la sequenza è definita come:

$$F_n = \begin{cases} 1 & \text{if } n = 1 \text{ or } n = 2 \\ F_{n-1} + F_{n-2} & \text{otherwise} \end{cases}$$

Questa definizione apparentemente semplice nasconde interessanti sfide computazionali.



La formula matematica di Fibonacci

Esiste una formula chiusa ben definita per calcolare direttamente il numero di Fibonacci F_n:

$$F_n = \frac{1}{\sqrt{5}}(\phi^n - \hat{\phi}^n)$$

Il problema dell'irrazionalità

Eppure, non è possibile calcolare F_n su una macchina usando questa formula a causa dell'irrazionalità dei due caratteri ϕ^n e $\hat{\phi}^n$. I numeri irrazionali richiederebbero una precisione infinita per essere rappresentati esattamente.

PHI = SEZIONE AUREA

Sono necessarie, quindi, altre soluzioni algoritmiche che possano essere implementate su computer reali con precisione finita.

Prima soluzione: Algoritmo ricorsivo

È possibile convertire la formula matematica in codice utilizzando un approccio ricorsivo. Ecco un esempio scritto in Python:

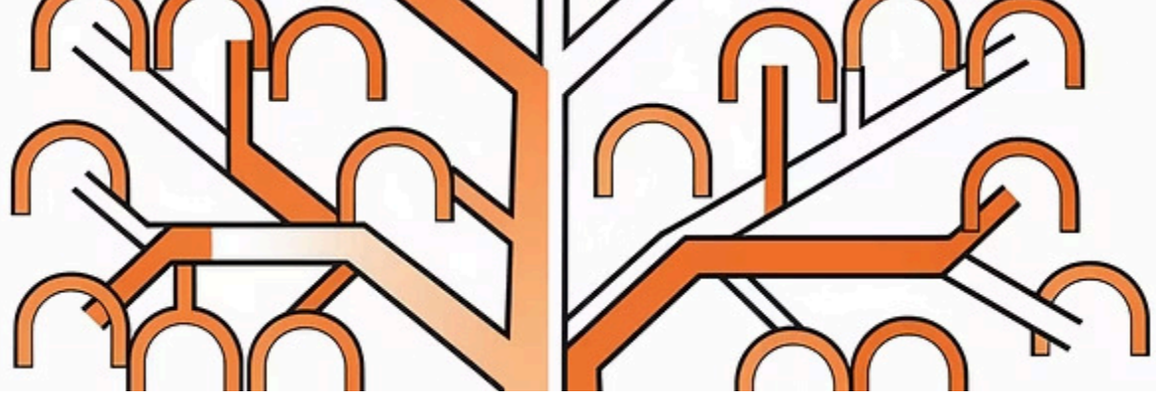
```
def recursiveFib(n):
    if n <= 2:
        return 1
    return recursiveFib(n - 1) + recursiveFib(n - 2)
```

Questo codice traduce direttamente la definizione matematica ricorsiva in un'implementazione software.

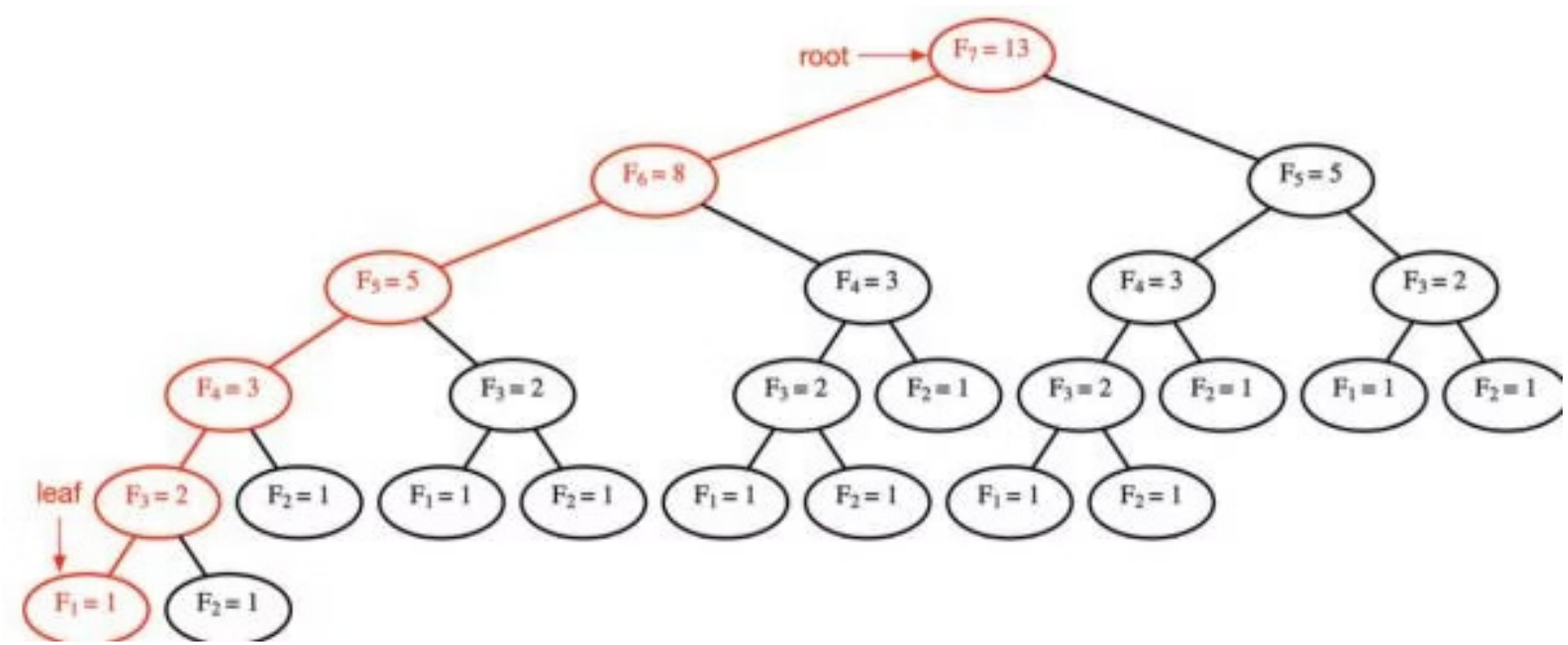
Definizione matematica

$$F_n = \begin{cases} 1 & n \leq 2 \\ F_{n-1} + F_{n-2} & n > 2 \end{cases}$$

La bellezza della ricorsione sta nella sua eleganza: il codice rispecchia perfettamente la definizione matematica.



L'albero ricorsivo di Fibonacci



Le chiamate ricorsive sono gestite dallo stack. Ad un qualsiasi tempo T, l'utilizzo di memoria dipende da tutte le chiamate attive al tempo T.

Il path massimo di chiamate ricorsive è legato al path leaf-root più lungo dell'albero di ricorsione. Osservando l'albero, notiamo che molti sottoproblemi vengono calcolati ripetutamente, suggerendo una potenziale inefficienza.

ANALISI DELLE PRESTAZIONI

Come misurare il tempo di esecuzione?

1

Secondi?

Troppo dipendente dalla macchina su cui l'algoritmo viene eseguito

2

Numero di istruzioni?

Troppo dipendente dall'architettura hardware e dal compilatore

3

Operazioni elementari ✓

Misura indipendente dalla macchina che cattura la complessità intrinseca

Contare il numero delle operazioni elementari è il metodo standard per analizzare gli algoritmi in modo astratto e generale.

Analisi del tempo: Algoritmo ricorsivo

```
function FIB2(INT n) → INT
  if n ≤ 2 then
    return 1
  else
    return FIB2(n - 1) + FIB2(n - 2)
```

Da questo pseudocodice possiamo estrarre la seguente relazione di ricorrenza:

$$T(n) = \begin{cases} 1 & n \leq 2 \\ T(n-1) + T(n-2) + 1 & n > 2 \end{cases}$$

Dove T(n) rappresenta il numero di operazioni elementari necessarie per calcolare F_n.

Risoluzione della ricorrenza

Analizziamo passo dopo passo la complessità dell'algoritmo ricorsivo:

$$T(n) = T(n-1) + T(n-2) + 1$$

01

Poiché $T(n-1) \geq T(n-2)$, abbiamo $T(n) \geq 2T(n-2) + 1$

02

Continuando: $T(n) \geq 4T(n-4) + 2 + 1$

03

Poi: $T(n) \geq 8T(n-6) + 4 + 2 + 1$

04

Generalizzando al k-esimo passo: $T(n) \geq 2^k T(n-2k) + \sum_{i=0}^{k-1} 2^i$

05

Fermandoci quando $n - 2k = 0 \Rightarrow k = \lfloor n/2 \rfloor$

06

Otteniamo: $T(n) \geq 2^{\lfloor n/2 \rfloor}$

Questo dimostra che l'algoritmo ricorsivo ha una complessità esponenziale!

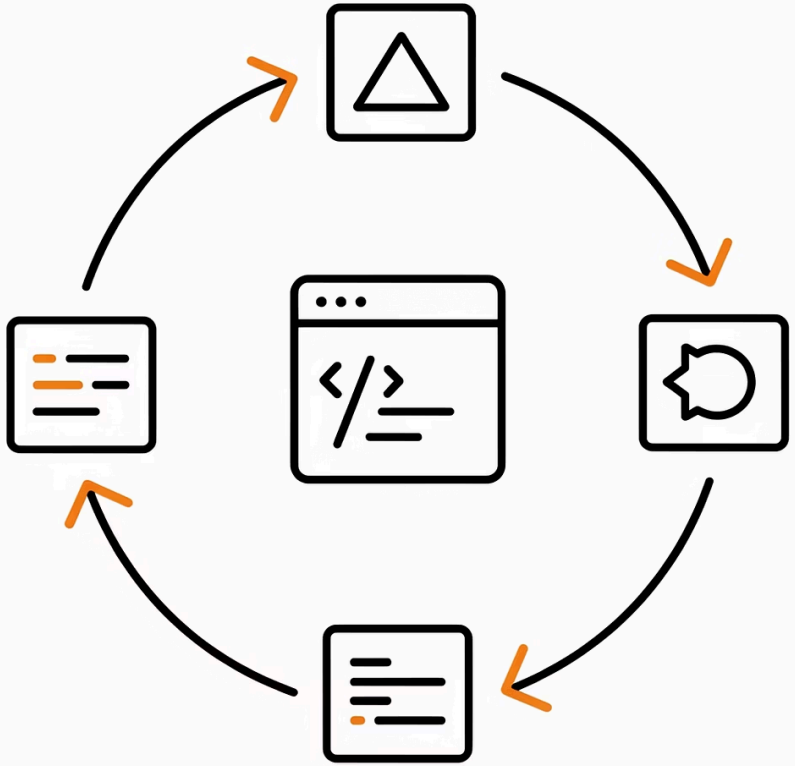
Seconda soluzione: Algoritmo iterativo

Un approccio molto più efficiente utilizza un ciclo iterativo. Consideriamo il seguente codice Python:

```
def iterativeFib(n):
    L = [0, 1, 1]
    for i in range(3, n + 1):
        L.append(L[i - 1] + L[i - 2])
    return L[-1]
```

- **Creazione nuova lista**: 1 operazione
- **Assegnazione L[0], L[1] e L[2]**: 3 operazioni
- **Iterazione (n-2) volte**: 3 operazioni per iterazione (somma + assegnamento + append)

Totale: $4 + 3(n - 2) = 3n - 2$ operazioni elementari (per $n > 2$). Un miglioramento drammatico rispetto all'approccio ricorsivo!

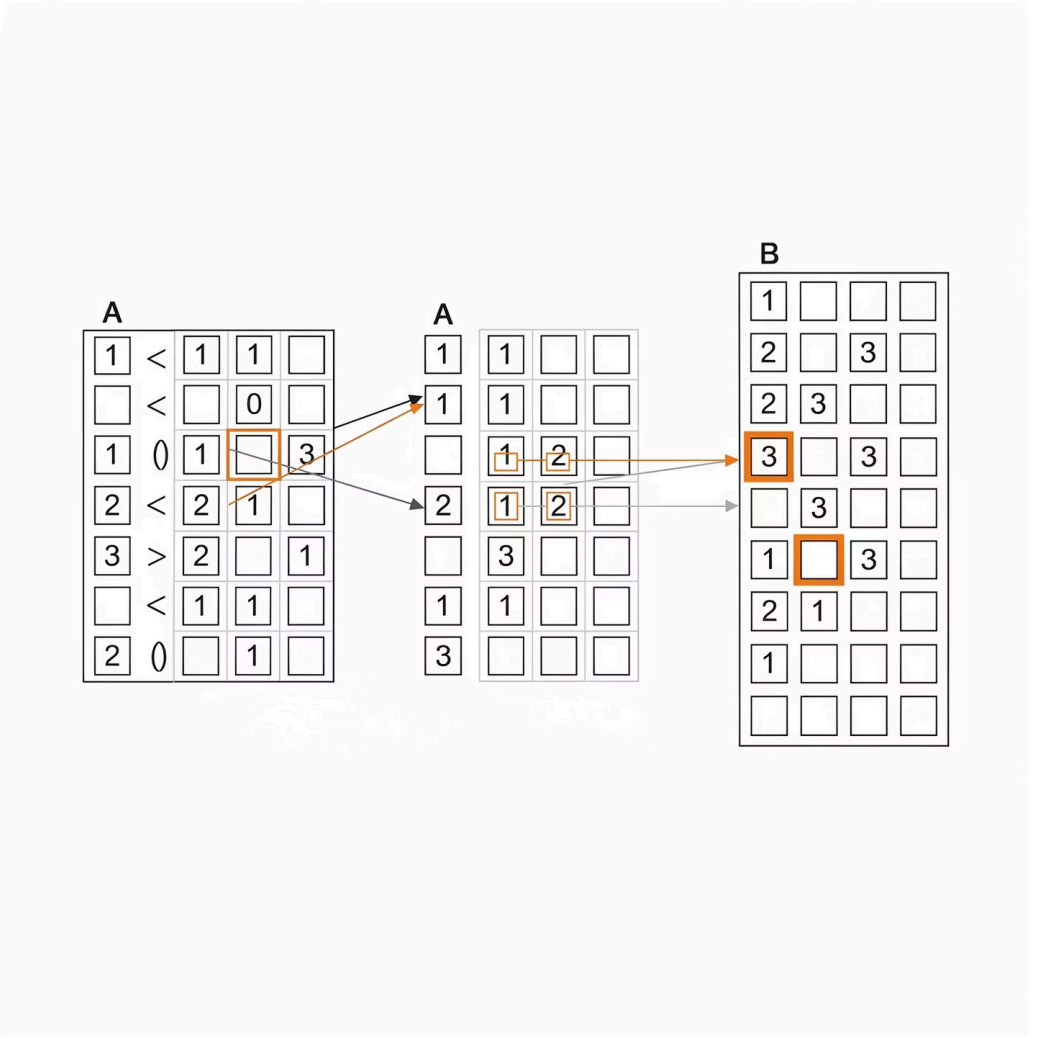


Terza soluzione: Calcolo matriciale

Esiste una soluzione ancora più efficiente basata sul calcolo matriciale. Consideriamo il seguente teorema, dimostrabile per induzione:

$$A^{n-1} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^{n-1} = \begin{pmatrix} F_n & F_{n-1} \\ F_{n-1} & F_{n-2} \end{pmatrix}$$

Dove $A = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}$ e la formula vale per ogni $n \geq 2$.



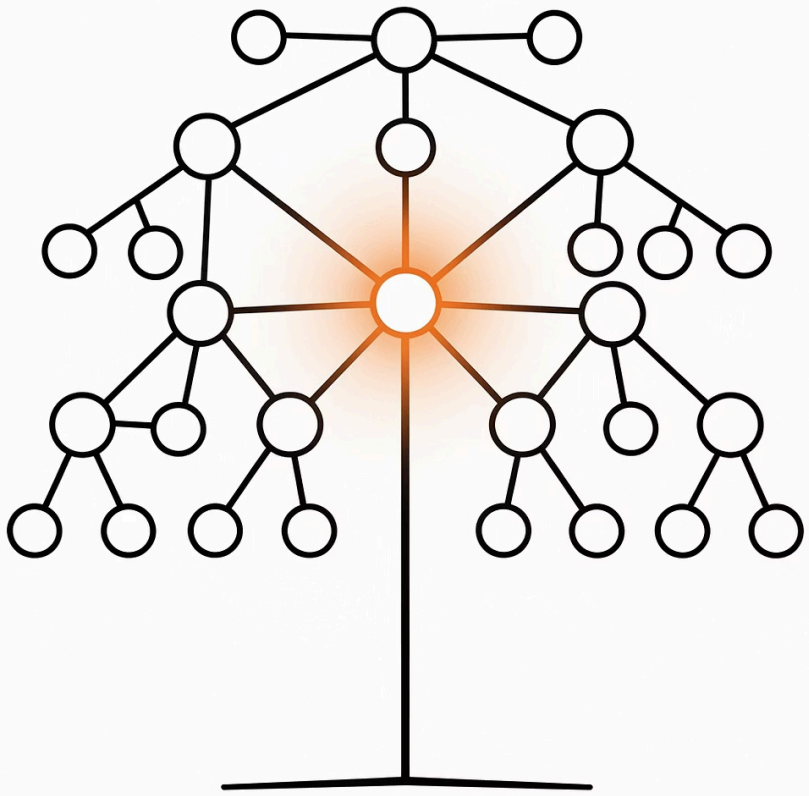
Ottimizzazione con elevamento a potenza veloce

La chiave dell'efficienza sta nell'osservazione che se n è pari:

$$A^n = (A^{n/2})^2, \quad A^{n+1} = (A^{n/2})^2 \cdot A$$

È possibile scrivere un algoritmo con tempo proporzionale a $\log(n)$, in quanto è necessario effettuare solo $\log(n)$ chiamate ricorsive per calcolare A^n.

Questo approccio divide ripetutamente il problema a metà, una tecnica fondamentale chiamata *divide et impera*.



Ma cosa vuol dire "tempo proporzionale a log(n)"?

Per comprendere davvero l'efficienza degli algoritmi, dobbiamo introdurre strumenti formali per descrivere e confrontare le loro prestazioni asintotiche.

TEORIA DELLA COMPLESSITÀ

Cost Function - Definizione

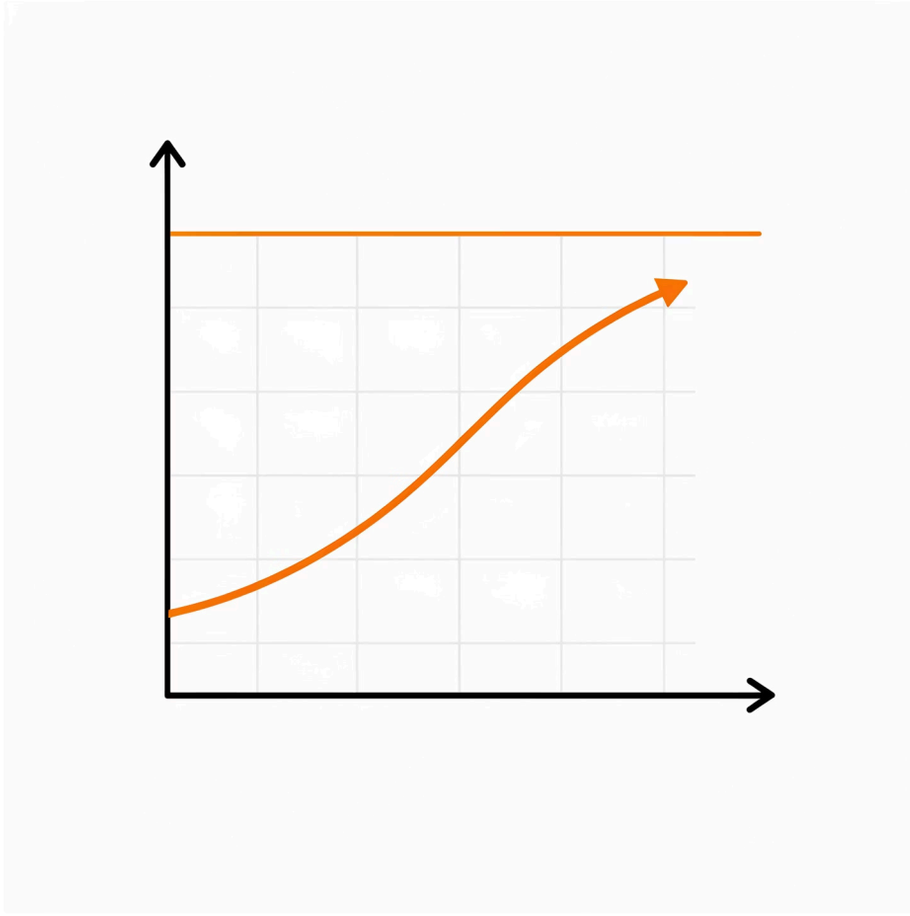
Dato un qualsiasi $n \geq 0$, definiamo $f(n) \geq 0$ come l'ammontare di risorse (in termini di tempo e di memoria) necessario per l'esecuzione di un generico algoritmo, dato un input di dimensione n.

In generale

Siamo interessati a valutare il **tasso di crescita** della cost function $f(n)$.

In particolare

Si va a considerare il **comportamento asintotico** della cost function $f(n)$, cioè come si comporta per valori molto grandi di n.



Questo ci permette di rispondere alla domanda: *"quale algoritmo ha performance migliori in termini di tempo di esecuzione?"*

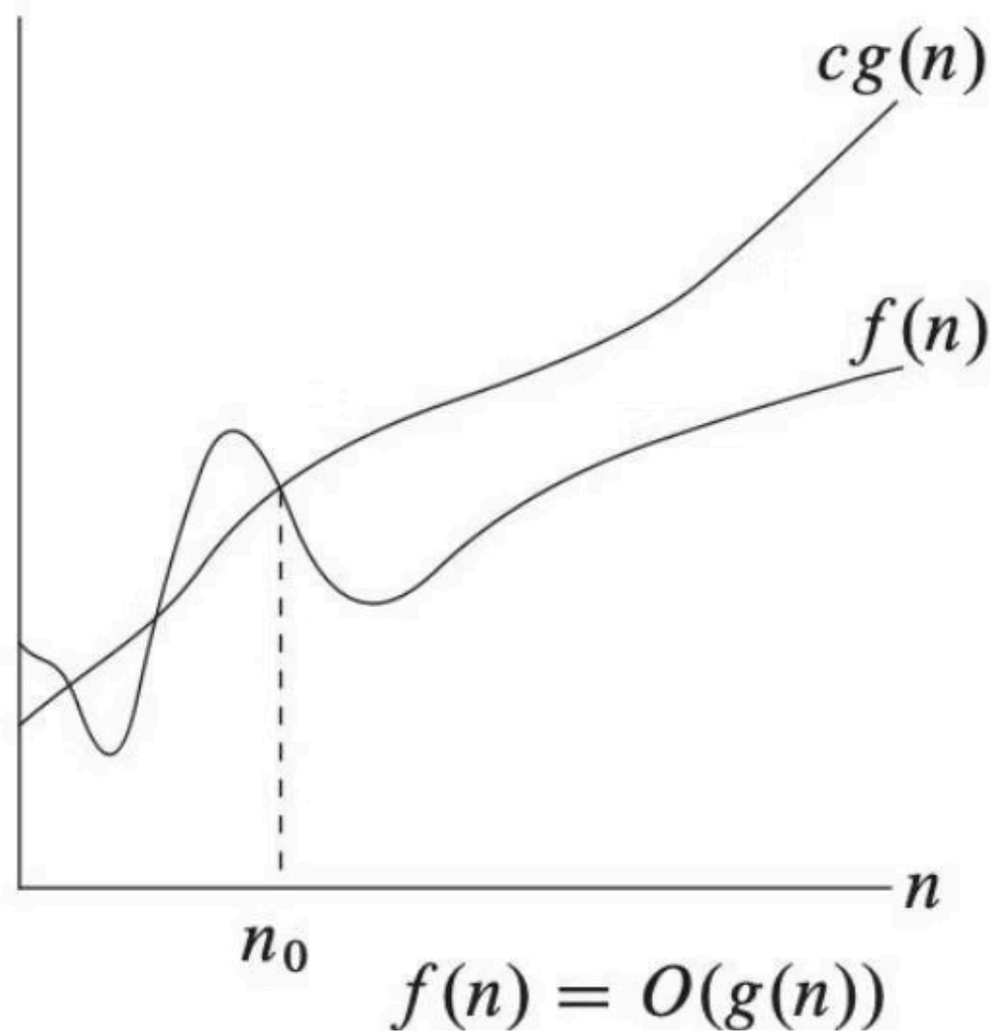
Big-O Notation - Definizione

Data una cost function $g(n)$, si definisce l'insieme O-grande come:

$$O(g(n)) = \{f(n) \mid \exists c > 0, n_0 \geq 0 \text{ such that } \forall n \geq n_0, f(n) \leq cg(n)\}$$

In altre parole, $f(n) = O(g(n))$ significa che $g(n)$ è un **limite superiore asintotico** per $f(n)$.

Esiste una costante c e un punto n_0 oltre il quale $f(n)$ è sempre minore o uguale a $c \cdot g(n)$.



Altre notazioni asintotiche

Big-Omega (Ω)

Limite inferiore asintotico - identifica comportamenti con tasso di crescita maggiore o uguale

Theta (Θ)

Limite stretto - identifica comportamenti con tasso di crescita esattamente uguale, sia superiormente che inferiormente

Little-o (o)

Limite superiore stretto - identifica comportamenti con tasso di crescita strettamente minore

Little-omega (ω)

Limite inferiore stretto - identifica comportamenti con tasso di crescita strettamente maggiore

In questo corso utilizzeremo principalmente la notazione Big-O, che è la più comune nell'analisi algoritmica pratica.

Ordini di crescita comuni

Ordine di crescita	Nome
$O(1)$	Costante
$O(\log n)$	Logaritmico
$O(\log^k n)$	Polilogaritmico, $k \geq 1$
$O(n^k)$, $0 < k < 1$	Sublineare
$O(n)$	Lineare
$O(n \log n)$	Pseudolineare (o linearitmico)
$O(n^k)$, $k > 1$	Polinomiale
$O(n^2)$	Quadratico (per $k = 2$)
$O(n^3)$	Cubico (per $k = 3$)
$O(c^n)$, $c > 1$	Esponenziale, base c
$O(n!)$	Fattoriale
$O(n^n)$	Esponenziale, base n

Confronto visivo dei tassi di crescita

Il grafico mostra come diverse funzioni crescono all'aumentare di n. Notate le differenze drammatiche:

Crescita lenta

Funzioni logaritmiche e costanti crescono molto lentamente - ideali per algoritmi efficienti

Crescita moderata

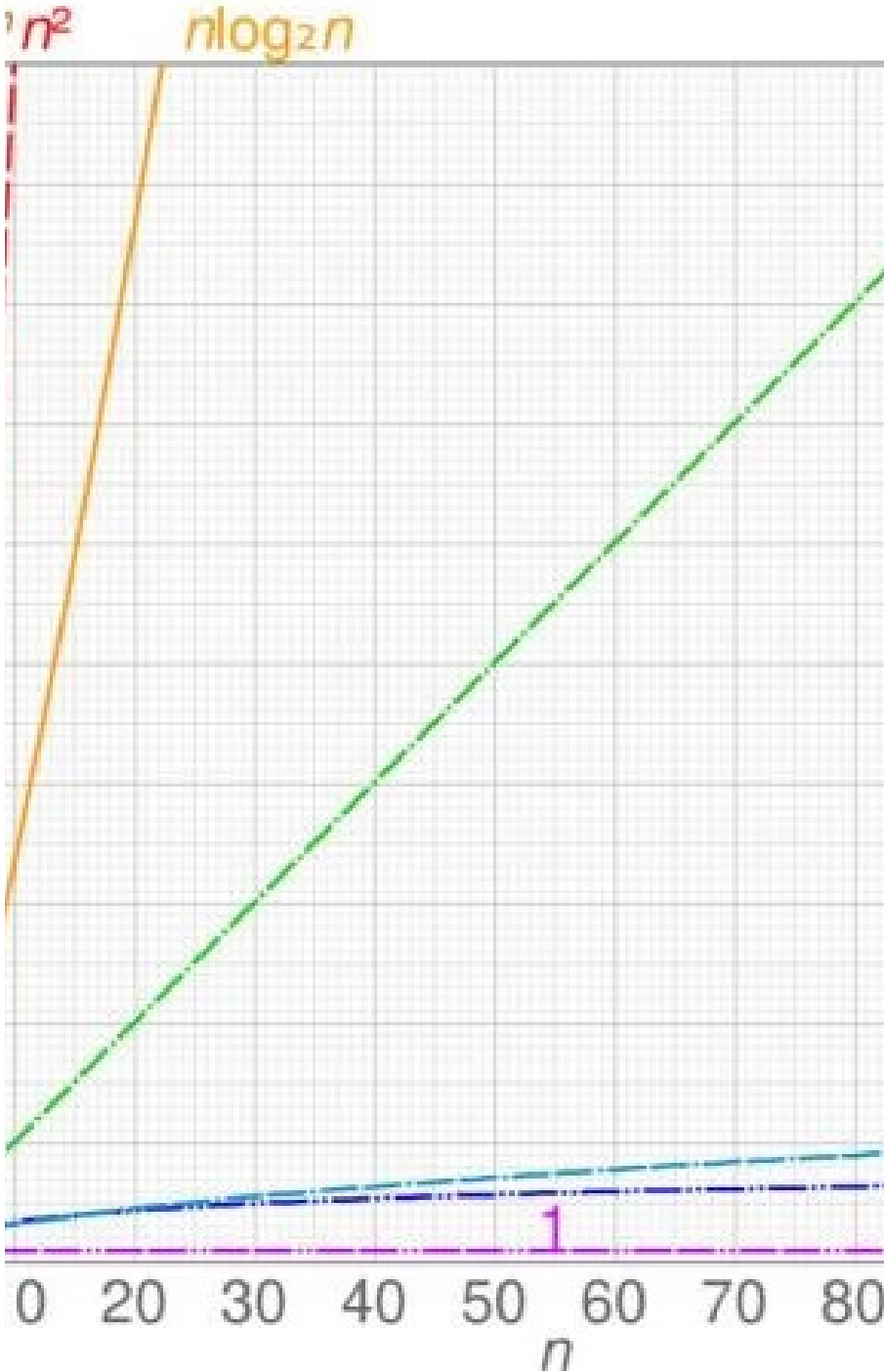
Funzioni lineari e pseudolineari sono accettabili per la maggior parte delle applicazioni pratiche

Crescita rapida

Funzioni polinomiali di grado alto diventano rapidamente impraticabili per input grandi

Crescita esplosiva

Funzioni esponenziali e fattoriali sono utilizzabili solo per input molto piccoli



ANALISI PRATICA

Best, Average e Worst Case

Best Case

Descrive il comportamento algoritmico sotto condizioni ottimali. Ad esempio, in una ricerca lineare l'elemento da cercare è il primo della lista.

Worst Case

Descrive il comportamento algoritmico sotto condizioni non favorevoli. Ad esempio, in una ricerca lineare l'elemento da cercare è l'ultimo della lista o non è presente.

Average Case

Descrive il comportamento algoritmico medio su tutti gli input possibili. Ad esempio, il tempo medio necessario per la ricerca lineare, assumendo che l'elemento sia distribuito uniformemente.

Nella pratica, l'analisi del worst case è spesso la più importante perché fornisce garanzie sul comportamento dell'algoritmo in qualsiasi situazione. L'average case è utile per valutare le prestazioni tipiche, ma richiede assunzioni sulla distribuzione degli input.